communityclient 插件深度审查报告

> 版本：1.0.0 | 类型：前端插件 | 代码量：~900 行
>
> 审查日期：2026-04-03 | 修复日期：2026-04-03

综合评分

维度	初审评分	修复后评分	简评
功能性	5/10	5/10	API 层完整，但大量后端功能无对应 UI
安全性	4/10	**6/10**	Token 改走 AuthController，AuthRefreshInterceptor 已接入，401 刷新链路完整
架构设计	7/10	7/10	GetX 分层清晰，DI 设计合理
正确性	4/10	**7/10**	全部 P0 Bug 已修复
错误处理	2/10	2/10	静默失败问题尚未处理
可测试性	2/10	2/10	Controller 仍耦合单例，待重构
总评	**4/10**	**6/10**	P0 正确性 + P1 安全性全部关闭，质量/可测试性项待后续处理

一、功能性分析

API 层 vs UI 层覆盖对比

功能	API 方法 ✅	Controller 🔄	UI 实现 ✅
获取文章列表	✅	✅	✅
创建/编辑文章	✅	✅	✅
删除文章	✅	❌	❌
文章详情	✅	✅	✅（简陋）
评论列表	✅	✅	✅
发表评论	✅	✅	✅
删除评论	✅	❌	❌
评论点赞	❌	❌	❌
文章点赞	✅	✅	✅
关注用户	✅	❌	❌
举报文章	✅	❌	❌
不喜欢文章	✅	❌	❌
阅读标记	❌	❌	❌
媒体上传	❌	❌	❌
黑名单	❌	❌	❌
屏蔽分类	❌	❌	❌
分类选择	✅（API）	❌	❌
我的草稿	❌	❌	❌
我的文章	❌	❌	❌

API 层对后端已实现的大部分接口做了封装，但只有不到一半的功能走通了 **Controller → UI** 这条链。

二、Bug 批判

🔴 Bug 1：文章列表将媒体数量误显示为点赞数

```
// articles_list_view.dart  line 144
Row(
  children: [
    const Icon(Icons.thumb_up_outlined, size: 14, color: Colors.grey),
    const SizedBox(width: 2),
    Text('${article.media.length}',  // ← 点赞图标，却显示媒体数量
        style: theme.textTheme.labelSmall),
  ],
),
```

`Article` proto 中有 `repeated Media media` 字段，用 `media.length` 作为点赞数完全错误。正确做法应从 `getArticleDetail` 的 `likelist.length` 获取，或后端在 Article 上冗余一个 `likeCount` 字段。

快速修复：当前 `GetArticlesResponse` 中的 `Article` 没有 likeCount，短期内可以隐藏点赞数或请求后端在 proto 中补充字段。

🔴 Bug 2：评论展示忽略接口已返回的用户名

```
// article_detail_view.dart
class _CommentTile extends StatelessWidget {
  final String content;
  final int userId;

  @override
  Widget build(BuildContext context) {
    // ...
    Text('用户 $userId',   // ← 硬编码拼接，即使接口已返回 userName
        style: Theme.of(context).textTheme.labelSmall),
```

后端 `GetCommentsResponse.Comment` 中已包含 `userName` 和 `userImageUrl`，前端完全忽略，改为显示 `用户 123` 这样的占位文本。用户体验很差。

🟠 Bug 3：文章详情路由的 `userId` 丢失

```
// articles_list_view.dart — ArticleCard.onTap
Get.toNamed(
  CommunityRoutes.articleDetail,
  arguments: {'articleId': article.articleID},
  // 没有传 userId！
);
```

`ArticleDetailView` 中从 `arguments` 读取 `userId`，默认值为 `0`：

```
final int userId = args['userId'] as int? ?? 0;
```

`userId = 0` 导致：

- 无法正确判断当前用户是否已点赞（`isLiked` 永远为 false）
- 无法显示文章作者信息（`getArticleDetail` 以 userID=0 查询）
- 无法判断是否显示编辑按钮（`a.userID == userId` → `0 == userId` 永远为 false）

🟠 Bug 4：`_ensureInit` 的重初始化逻辑是空操作

```
// community_api_service.dart
void _ensureInit() {
  if (_initialized) return;
  _dio = Dio(BaseOptions(baseUrl: _baseUrl, ...));
  // ...
  _initialized = false; // defer re-init when baseUrl changes
  _initialized = true;
}
```

注释写"baseUrl 变化时 defer re-init"，但代码逻辑是：先设 `false`，立即设 `true`，完全等价于直接设 `true`。当 `baseUrl` 发生变化（如环境切换），Dio 实例永远不会重建，新的 baseUrl 不会生效。

🟡 Bug 5：`ArticleDetailView` 有条件懒注册 Controller

```
// article_detail_view.dart
if (!Get.isRegistered<ArticleDetailController>()) {
  Get.put(ArticleDetailController(articleId: articleId, userId: userId));
}
```

View 中做注册是反模式，且检查 `isRegistered` 带来副作用：从文章 A 进入详情，再返回，再进入文章 B，`ArticleDetailController` 已注册，懒注册被跳过，显示的仍是文章 A 的数据。

三、安全性批判

🟠 Token 明文存储于 SharedPreferences

```
// community_api_service.dart
final prefs = await SharedPreferences.getInstance();
final token = prefs.getString('accessToken') ?? '';
```

`SharedPreferences` 在 Android 上以明文 XML 存储，Root 用户或备份攻击可直接读取 access token。更安全的做法是使用 `flutter_secure_storage`（底层 Android Keystore / iOS Keychain）。

🟡 评论输入无长度限制

```
// article_detail_view.dart
TextField(
  controller: _commentCtrl,
  decoration: const InputDecoration(hintText: '写评论…'),
  // 无 maxLength，无 maxLines 截断
)
```

用户可以提交任意长度评论，后端也没有对 `content` 做长度校验，超长内容会直接插入数据库。

🟡 Token 过期后无刷新机制，401 静默失败

`CommunityApiService` 的请求拦截器仅从 `SharedPreferences` 读取 `accessToken` 并携带到请求头，没有任何 token 刷新逻辑：

```
// community_api_service.dart（onRequest 拦截器）
final prefs = await SharedPreferences.getInstance();
final token = prefs.getString('accessToken') ?? '';
if (token.isNotEmpty) {
  options.headers['Authorization'] = 'Bearer $token';
}
```

相比之下，`authclient` 的 Dio 实例配置了完整的 `AuthRefreshInterceptor`，token 过期时自动用 refresh token 换新 access token 并重试原请求。

`communityclient` 缺少这一层：token 过期 → 服务端返回 401 → `catch` 块仅打 warning 日志 → 用户看到永久加载或空白页面。与现有的静默错误处理问题叠加，故障完全不可见。

建议：将 `authclient` 的刷新拦截器提取为共享工具包，在 `_ensureInit` 中注入到 `communityclient` 的 Dio 实例。

四、架构设计批判

问题 1：Controller 强耦合 `CommunityApiService` 单例，无法 Mock

```
class CommunityHomeController extends GetxController {
  final _api = CommunityApiService();  // 直接拿单例，无法注入
```

所有 Controller 直接引用 `CommunityApiService()` 单例工厂，无法在测试中替换为 Mock 实现，这也是为什么 communityclient 没有任何测试。

改良方案：通过构造函数注入：

```
class CommunityHomeController extends GetxController {
  final CommunityApiService _api;
  CommunityHomeController({required this.userId, CommunityApiService? api})
    : _api = api ?? CommunityApiService();
```

问题 2：proto 枚举值以魔数方式硬编码

```
// community_home_controller.dart
// ArticleClassification (从 community.proto): discover=2, recommended=1,
follow=3
static const int _clsDiscover    = 2;
static const int _clsRecommended = 1;
static const int _clsFollow      = 3;
```

注释本身证明这些是 proto 枚举的数值，应该直接用 `ArticleClassification.discover.value` 等（proto 生成的 Dart 代码中枚举有 `.value` 属性），避免与后端枚举定义脱钩。

问题 3：`articleCreate` Binding 中丢失 `existingArticle`

```
// community_routes.dart
GetPage(
  name: articleEdit,
  page: () => const ArticleCreateView(),
  binding: ArticleCreateBinding(userId: userId),  // 没有传 existingArticle
),
```

`ArticleCreateBinding` 注入的 `ArticleCreateController` `existingArticle` 永远为 null，编辑文章时无法加载已有内容到编辑器。

五、错误处理批判

全部 IO 操作静默吞掉异常

```
// 三个 Controller 中重复同样的模式
Future<void> loadDiscover({bool refresh = false}) async {
  // ...
  try {
    final resp = await _api.getArticles(req);
    // ...
  } catch (e) {
    _log.warning('loadDiscover error: $e');  // 仅记日志，界面无任何反馈
  }
}
```

网络断开、服务器 500、token 过期……所有错误的处理方式均为"打个 warning 日志，假装一切正常"。用户看到的是永久转圈或空白页面，没有任何错误提示、重试按钮或 fallback UI。

改良方案：引入错误状态并在 UI 中响应：

```
final errorMsg = Rxn<String>();

} catch (e) {
  _log.warning('loadDiscover error: $e');
  errorMsg.value = '加载失败，请检查网络';
}

// UI
Obx(() {
  if (ctrl.errorMsg.value != null) {
    return ErrorView(
      message: ctrl.errorMsg.value!,
      onRetry: () => ctrl.loadDiscover(refresh: true),
    );
  }
  // ...
})
```

六、改良建议总结

优先级	改良项	预估工作量	状态
P0	修复 ArticleCard 点赞数显示（`media.length` → 实际 likeCount）	1h	✅ 已修复
P0	修复 `articles_list_view` 跳转详情时不传 `userId`	0.5h	✅ 已修复
P0	修复 `ArticleDetailController` 懒注册导致切换文章数据不更新	1h	✅ 已修复
P0	评论展示使用接口返回的 `userName` / `userImageUrl`	1h	✅ 已修复
P0	修复编辑文章路由不传 `existingArticle`	1h	✅ 已修复
P1	修复 `_ensureInit` 无效的 re-init 逻辑	0.5h	✅ 已修复
P1	全部 catch 块补充 UI 错误状态和重试机制	4h	⬜ 待处理
P1	Token 改用 `flutter_secure_storage` 存储	2h	✅ 已修复（改走 AuthController，宿主通过 IAuthStorage 控制存储实现，communityclient 不再直接访问 SharedPreferences）
P1	接入 token 刷新拦截器（复用 authclient 的 `AuthRefreshInterceptor`），消除 401 静默失败	2h	✅ 已修复（AuthRefreshInterceptor + TokenInterceptor 已注册，支持预刷新 + 401 重试 + 并发防护）
P1	Controller 构造函数注入 API 服务，解除单例耦合	2h	⬜ 待处理
P2	用 proto 枚举值替代魔数常量	0.5h	⬜ 待处理（proto 暂无枚举定义）
P2	评论输入框加 `maxLength`（300 字）	1h	✅ 已修复
P3	实现删除文章 / 删除评论 UI	3h	⬜ 待处理

P3	实现关注用户 UI	2h	⬜ 待处理
P3	实现 "我的文章 / 草稿" 列表	4h	⬜ 待处理

七、修复记录

> 修复日期：2026-04-03

✅ P0-1：点赞数显示修复

文件：`articles_list_view.dart`

`article.media.length` 更正为 `article.likeCount`（Article proto 已有该字段）。

✅ P0-2：`ArticleCard` 导航缺少 `userId`

文件：`articles_list_view.dart`、`community_home_view.dart`

- `ArticlesListView` 新增 `required int userId` 参数
- `_ArticleCard` 接收 `userId` 并在 `Get.toNamed` 的 `arguments` 中一并传递
- `CommunityHomeView` 传入 `userId: ctrl.userId`

✅ P0-3：`ArticleDetailController` 懒注册反模式

文件：`community_routes.dart`、`article_detail_view.dart`

- `articleDetail` 路由改用 `BindingsBuilder`，在每次导航时从 `Get.arguments` 读取最新 `articleId` + `userId` 并 `Get.put` 全新 Controller 实例
- `ArticleDetailView` 移除了视图内 `isRegistered` 判断 + 懒注册代码，改为直接 `Get.find<ArticleDetailController>()`

✅ P0-4：评论显示用户名/头像

文件：`article_detail_view.dart`

- `_CommentTile` 改为接收完整 `Comment` proto 对象
- `displayName` 优先使用 `comment.userName`，为空时降级为 `用户 ${comment.userID}`
- `CircleAvatar` 有 `userImageUrl` 时展示网络图片，否则显示用户名首字母

✅ P0-5：编辑文章路由缺少 `existingArticle`

文件：`community_routes.dart`

- `articleEdit` 路由改用 `BindingsBuilder`，从 `Get.arguments['article']` 读取 `Article` 对象并传入 `ArticleCreateController`

✅ P1：`_ensureInit` 无效 re-init 逻辑

文件：`community_api_service.dart`

移除 `_initialized = false; // defer re-init` 这行无意义的赋值，逻辑变为正确的 `_initialized = true`。

✅ P2：评论输入 `maxLength`

文件：`article_detail_view.dart`

评论 `TextField` 添加 `maxLength: 300`，并设 `counterText: ''` 隐藏计数器 UI（Flutter 默认会显示 "xxx/300"）。

✅ P1-安全：Token 读取改走 AuthController

文件：`community_api_service.dart`、`pubspec.yaml`

- 移除直接依赖 `shared_preferences`，新增 `auth_client: path: ../authclient`
- `_ensureInit` 不再通过 `SharedPreferences.getInstance()` 读取 `'accessToken'` key（Android 明文 XML，可被 Root/备份攻击）
- 改为 `Get.find<AuthController>()` 获取当前 access/refresh token，存储安全性由宿主 App 的 `IAuthStorage` 实现控制（默认使用 `flutter_secure_storage`）

✅ P1-安全：接入 AuthRefreshInterceptor

文件：`community_api_service.dart`

- 注册 `AuthRefreshInterceptor`：token 即将过期时自动预刷新，401 时重试并附带内置并发防护（避免多请求同时触发刷新竞态）
- 注册 `TokenInterceptor`：每个请求自动注入 `Authorization: Bearer <token>` 头
- 刷新链路：`ApiConfig().authService.refreshToken(refreshToken)` → `capturedAuth.saveSession(token: newTokens)` → 返回新 access token
- `skipRoutes: const []`，所有社区接口均需鉴权